

RAPPORT TECHNIQUE

Mise en place d'une solution de Déploiement Continu basée sur Proxmox, Terraform et Ansible



MetalOps

Adresse postale:
Rue François Coli, 06210

DYSHLEVYY Bohdan
RETIERE Romain
HERBRECHT-BOURGOIN Clement

SOMMAIRE

1. Introduction	3
2. Contexte et portée du projet	4
3. Intégration du DevOps dans le projet	5
4. Architecture	6
4.1 Architecture système et réseau	6
4.2 Hardware (environnement matériel)	7
4.3 Operating System (choix du système)	8
5. Conception (outils mis en œuvre)	9
5.1 Déploiement de l'infrastructure : Proxmox + Terraform	9
5.2 Configuration des services : Ansible + Nginx	10
5.3 Supervision (infra et applicative)	11
6. Plan de développement et mise en œuvre	12
7. Résultats obtenus	13
8. Budget et ressources	15
9. Gestion de projet et organisation du travail	16
10. Axes d'amélioration	17
11. Conclusion	18
12. Annexes	19

1. Introduction

Dans un contexte où les entreprises doivent livrer de plus en plus vite des nouvelles fonctionnalités tout en garantissant stabilité, sécurité et reproductibilité, l'automatisation des infrastructures devient un besoin concret et non plus un "plus". Cette SAE Cloud s'inscrit dans cette réalité. Notre équipe a été missionnée pour intervenir dans une entreprise fictive en difficulté, dont les livraisons prennent du retard et dont les déploiements reposent sur des pratiques manuelles réalisées par l'équipe Ops. Les environnements sont montés "à la main", sans standard clair, sans traçabilité, et avec un risque d'erreurs élevé. À chaque nouvelle mise en production, la même série d'actions est répétée, parfois différemment selon la personne qui déploie, ce qui rend les incidents difficiles à diagnostiquer.

Le projet vise donc à transformer ce modèle artisanal en une solution moderne de déploiement continu, fondée sur les principes DevOps et l'Infrastructure as Code. Le but n'est pas uniquement d'installer des outils, mais de mettre en place une chaîne cohérente qui permet de créer une infrastructure de manière reproductible, puis de configurer automatiquement les services applicatifs, avec un résultat identique à chaque exécution. Les bénéfices attendus sont immédiats : réduction du temps de déploiement, diminution des erreurs humaines, amélioration de la documentation technique grâce au code versionné, et meilleure collaboration entre développement et exploitation.

Les principales parties prenantes dans ce scénario sont l'équipe DevOps nouvellement embauchée (nous), l'équipe Ops qui effectuait jusqu'ici les déploiements manuellement, et l'ingénieur DevOps "Jean-Paul" orienté CI, déjà présent dans l'entreprise mais qui ne couvre pas la partie CD. L'objectif final est de démontrer qu'un déploiement automatique et reproductible est possible, et qu'il apporte une valeur directe à l'entreprise, notamment en fiabilité et en rapidité de livraison.

2. Contexte et portée du projet

À notre arrivée dans l'entreprise, l'organisation est clairement cloisonnée. Les développeurs produisent du code et les builds sont déjà automatisés (une forme d'intégration continue existe), mais la mise en production reste dépendante des opérations. Or, chaque nouveau déploiement implique la création manuelle d'une machine virtuelle, l'installation du système d'exploitation, la configuration réseau, l'installation des services, puis des tests manuels. Ces étapes prennent du temps, mobilisent les équipes et introduisent des variations entre environnements. Deux serveurs supposés identiques peuvent diverger sur un détail (paquets, configuration réseau, DNS, version d'un service), ce qui génère des dysfonctionnements parfois invisibles au début et coûteux à corriger ensuite.

La portée du projet est de proposer et de mettre en œuvre une solution de déploiement continu au sens "infrastructure + configuration". Nous avons donc ciblé un périmètre volontairement réaliste mais maîtrisable dans le cadre d'une SAE : déployer automatiquement une infrastructure composée d'un Load Balancer et de deux serveurs Web, puis configurer automatiquement les services pour obtenir un site web disponible via l'IP du Load Balancer. Le but est de démontrer la reproductibilité : reconstruire l'environnement rapidement, à l'identique, simplement en relançant la chaîne.

Les limites du projet sont assumées. Nous ne mettons pas en place une supervision complète (même si on la mentionne comme amélioration future), nous ne réalisons pas un plan de reprise d'activité, et nous ne déployons pas une application "métier" complexe. En revanche, nous mettons en œuvre une architecture cohérente, une automatisation complète, et nous documentons les difficultés réelles rencontrées (réseau, NAT, DNS, stockage LVM, configuration Nginx), car ce sont précisément ces points qui font la différence entre une démo "qui marche une fois" et une solution reproductible.

3. Intégration du DevOps dans le projet

Le DevOps est souvent résumé à une combinaison d'outils, mais dans un cadre projet il doit être vu comme une approche d'organisation et d'amélioration continue. Dans notre cas, l'intégration du DevOps se traduit par trois axes concrets. Le premier axe est l'automatisation : on remplace les actions manuelles par des procédures reproductibles, exécutées par des outils standard. Le deuxième axe est la traçabilité : l'infrastructure n'est plus un ensemble de réglages faits dans une interface graphique et oubliés, elle est décrite dans des fichiers versionnés. Le troisième axe est la collaboration et la séparation claire des responsabilités : Terraform gère la création des ressources (provisionnement) et Ansible gère leur configuration (déploiement applicatif), ce qui rend la solution lisible et maintenable.

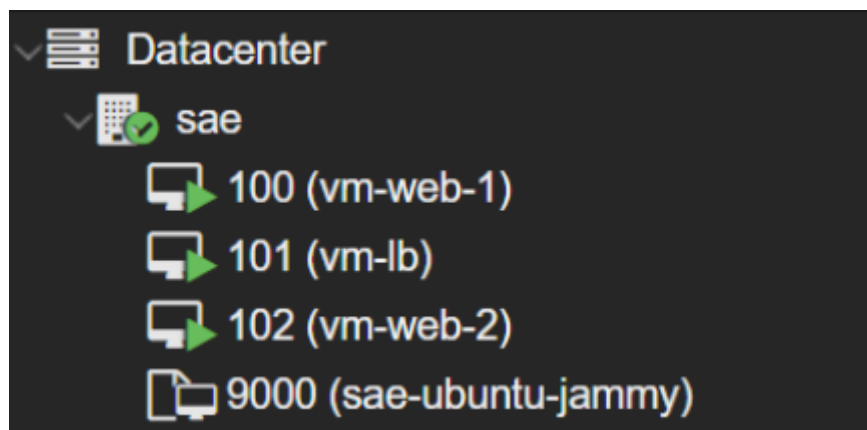
Cette approche apporte des avantages clairs dans notre projet. Elle réduit les délais, car recréer l'infrastructure ne demande plus une série d'actions manuelles. Elle fiabilise les déploiements, car la même exécution produit le même résultat. Elle rend les changements plus simples, car il est possible d'adapter un paramètre (taille disque, IP, bridge, configuration Nginx) et de rejouer le déploiement. Elle améliore la qualité de la documentation, car la documentation devient en grande partie le code lui-même.

Cependant, il existe aussi des inconvénients et des contraintes. Mettre en place l'automatisation demande un investissement initial plus important : comprendre l'API Proxmox, gérer cloud-init, comprendre les bridges réseau, diagnostiquer les erreurs de provisioning, et écrire des playbooks idempotents. Cette phase peut être plus longue qu'un déploiement "fait vite à la main", mais l'intérêt apparaît dès que l'on doit reconstruire, corriger ou industrialiser. Un autre point est que l'automatisation peut échouer pour des raisons "infrastructure" (stockage saturé, réseau mal routé, DNS absent) et il faut donc acquérir une vraie capacité de diagnostic. Justement, ce projet nous a permis d'apprendre à résoudre ces problèmes de manière méthodique.

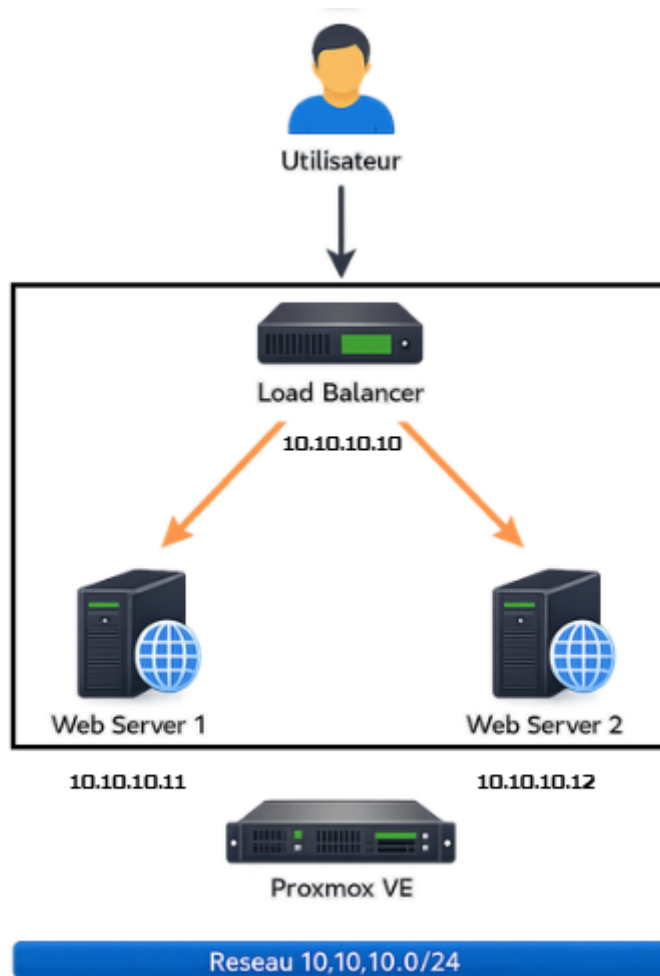
4. Architecture

4.1 Architecture système et réseau

L'architecture mise en place repose sur un hyperviseur Proxmox, lui-même exécuté dans un environnement VirtualBox. Sur Proxmox, nous avons déployé trois machines virtuelles Ubuntu Server 22.04 : une machine jouant le rôle de Load Balancer et deux machines jouant le rôle de serveurs Web. L'accès utilisateur se fait via le Load Balancer, qui redistribue ensuite les requêtes HTTP vers les deux serveurs Web grâce à Nginx configuré en reverse proxy avec un upstream.



Sur le plan réseau, l'architecture est basée sur un réseau interne isolé en 10.10.10.0/24, utilisé par les VMs applicatives. Pour permettre aux VMs d'installer des paquets (apt), un mécanisme de routage/NAT est nécessaire afin qu'elles puissent accéder à Internet. Concrètement, nous avons mis en place un bridge interne (vmbri1) et configuré le forwarding IPv4 ainsi que la translation d'adresse, ce qui permet aux VMs du réseau interne d'atteindre l'extérieur.



Cette architecture est volontairement simple, mais elle est réaliste et extensible. On peut facilement ajouter un troisième serveur web, ajouter du HTTPS, ajouter des health checks, ou même brancher une supervision. Elle illustre surtout le concept important : l'infrastructure et la configuration sont reproductibles et peuvent être recréées à l'identique.

4.2 Hardware (environnement matériel)

L'environnement matériel repose sur une machine hôte exécutant VirtualBox. La machine virtuelle Proxmox a été configurée avec des ressources suffisantes pour héberger plusieurs VMs. Nous avons notamment rencontré une contrainte importante : la taille initiale du stockage LVM thin était trop faible. Après le déploiement de plusieurs machines virtuelles et l'utilisation de disques de 10 Go, le pool local-lvm est arrivé à saturation (100 %), ce qui a provoqué des erreurs disque et des statuts io-error sur certaines VMs. Cet incident a été un vrai point d'apprentissage, car il nous a forcés à comprendre la différence entre l'espace "vu" dans la VM Proxmox et l'espace réellement disponible dans le thin pool, ainsi que la nécessité d'anticiper la capacité.

Nous avons donc augmenté la taille du disque dédié au stockage, puis étendu la capacité du LVM thin afin de stabiliser la plateforme. Après cette correction, les VMs ont retrouvé un fonctionnement normal et nous avons pu poursuivre l'automatisation.

4.3 Operating System (choix du système)

Les machines applicatives utilisent Ubuntu Server 22.04. Ce choix est motivé par la stabilité de la distribution, la disponibilité des paquets, et surtout sa compatibilité avec cloud-init, qui est central dans notre solution. Cloud-init permet d'injecter automatiquement des paramètres au premier boot, notamment l'utilisateur, la clé SSH, et la configuration IP. Sur Proxmox, nous avons préparé un template Ubuntu (sae-ubuntu-jammy) afin de pouvoir cloner rapidement des VMs identiques avec Terraform. Ce template a été conçu pour être "propre", c'est-à-dire adapté à la duplication (mode template Proxmox), et exploitable dans un cycle IaC.

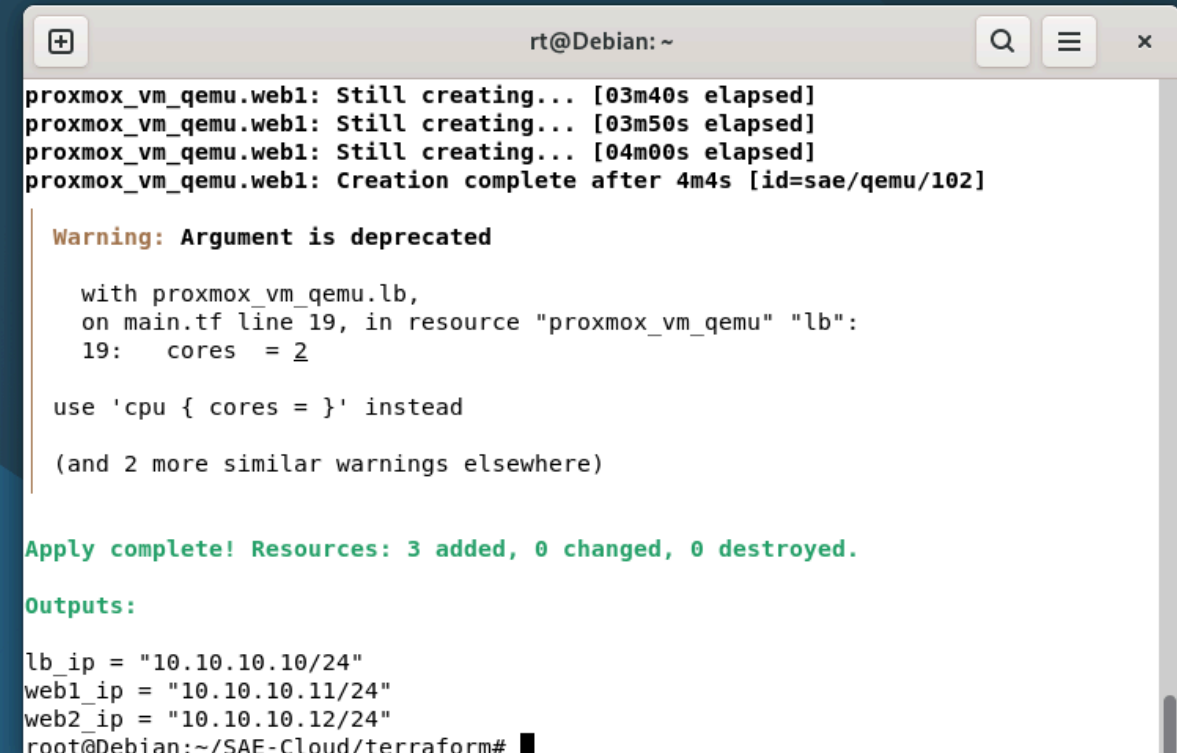
5. Conception (outils mis en œuvre)

5.1 Déploiement de l'infrastructure : Proxmox + Terraform

Proxmox est la couche de virtualisation. Il nous fournit une API, la gestion des VMs, des disques, du stockage LVM, ainsi que la gestion réseau via des bridges.

Terraform intervient au-dessus pour automatiser la création des VMs. L'idée est que l'infrastructure ne soit plus une suite de clics dans l'interface Proxmox, mais une description déclarative dans des fichiers versionnés.

Dans notre cas, chaque VM est décrite par des paramètres clairs : nom, CPU, RAM, stockage, bridge réseau, et configuration cloud-init (IP statique, gateway, clé SSH, utilisateur). Lorsque Terraform exécute `terraform apply`, il contacte l'API Proxmox, clone le template Ubuntu, applique les paramètres, crée les disques (dont le disque de 10 Go), et démarre les machines. Cette approche garantit que l'infrastructure peut être reconstruite à tout moment de manière identique. Elle rend également la solution plus robuste, car elle évite les différences manuelles entre les machines.



```
rt@Debian: ~  
proxmox_vm_qemu.web1: Still creating... [03m40s elapsed]  
proxmox_vm_qemu.web1: Still creating... [03m50s elapsed]  
proxmox_vm_qemu.web1: Still creating... [04m00s elapsed]  
proxmox_vm_qemu.web1: Creation complete after 4m4s [id=sae/qemu/102]  
  
Warning: Argument is deprecated  
  
    with proxmox_vm_qemu.lb,  
    on main.tf line 19, in resource "proxmox_vm_qemu" "lb":  
    19:   cores = 2  
  
    use 'cpu { cores = }' instead  
  
    (and 2 more similar warnings elsewhere)  
  
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
lb_ip = "10.10.10.10/24"  
web1_ip = "10.10.10.11/24"  
web2_ip = "10.10.10.12/24"  
root@Debian:~/SAE-Cloud/terraform#
```

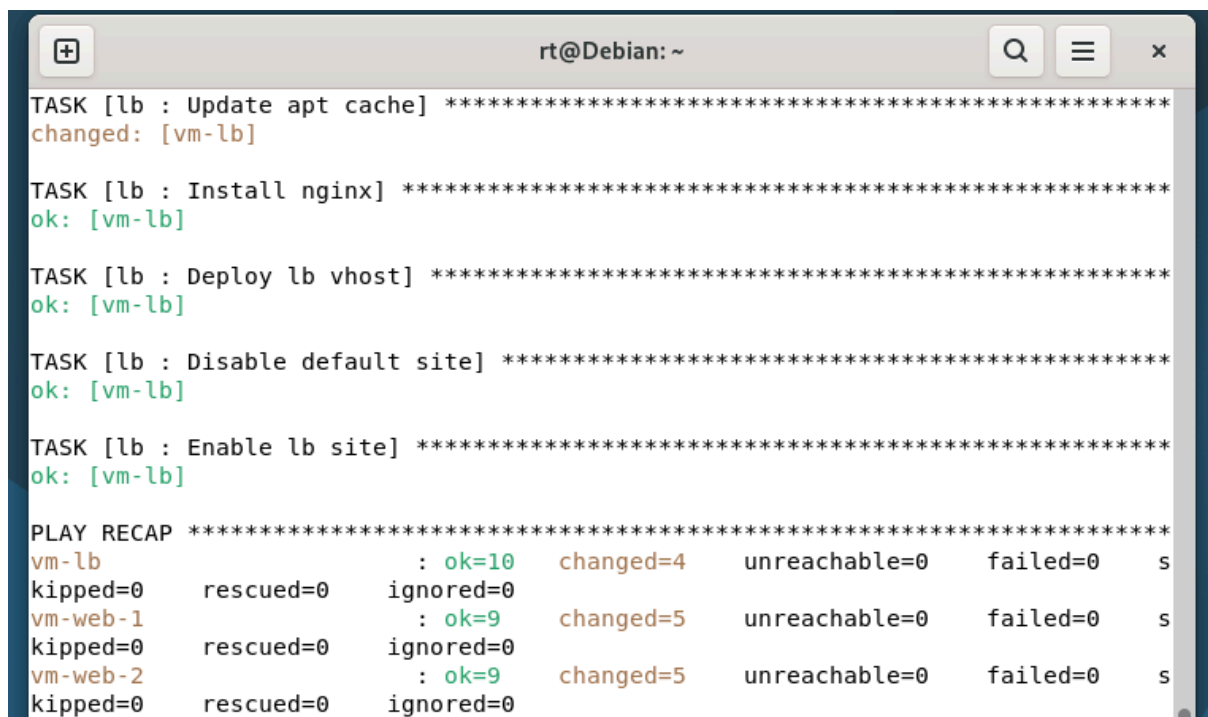
Un point important que nous avons appris ici est qu'un provisioning "réussi" ne dépend pas uniquement de Terraform. Il dépend aussi de l'état de l'infrastructure sous-jacente (stockage disponible, réseau cohérent, template cloud-init correct). Cela explique pourquoi certaines erreurs rencontrées au début (boot, ping

impossible, absence de cloud-init, ou erreurs disque) n'étaient pas des "bugs Terraform" mais des problèmes d'environnement.

5.2 Configuration des services : Ansible + Nginx

Une fois les machines créées, il faut les rendre utiles. Ansible intervient pour automatiser la configuration logicielle. Ansible se connecte en SSH, sans agent, ce qui est adapté à un environnement simple comme le nôtre. Les serveurs Web reçoivent Nginx, une page d'accueil personnalisée (affichant le hostname pour prouver quel serveur répond), et le service est activé au boot.

Le Load Balancer reçoit lui aussi Nginx, mais avec une configuration différente. Nous l'utilisons comme reverse proxy avec un bloc upstream, qui définit la liste des serveurs backend. Ensuite, un serveur Nginx écoute sur le port 80 et redirige les requêtes vers l'upstream. Cela permet d'obtenir un load balancing simple mais fonctionnel.



```
rt@Debian: ~  
TASK [lb : Update apt cache] *****  
changed: [vm-lb]  
  
TASK [lb : Install nginx] *****  
ok: [vm-lb]  
  
TASK [lb : Deploy lb vhost] *****  
ok: [vm-lb]  
  
TASK [lb : Disable default site] *****  
ok: [vm-lb]  
  
TASK [lb : Enable lb site] *****  
ok: [vm-lb]  
  
PLAY RECAP *****  
vm-lb : ok=10 changed=4 unreachable=0 failed=0 s  
kipped=0 rescued=0 ignored=0  
vm-web-1 : ok=9 changed=5 unreachable=0 failed=0 s  
kipped=0 rescued=0 ignored=0  
vm-web-2 : ok=9 changed=5 unreachable=0 failed=0 s  
kipped=0 rescued=0 ignored=0
```

Une difficulté réelle a été la cohérence des IP. Après un changement de plan d'adressage (passage vers le réseau interne 10.10.10.0/24), la configuration Nginx du Load Balancer pointait encore vers les anciennes IP 192.168.1.x. Résultat : Nginx retournait un 502 Bad Gateway, car il n'arrivait pas à joindre les backends. La résolution a été de corriger la configuration upstream pour utiliser 10.10.10.11 et 10.10.10.12, puis de recharger Nginx. Ce point illustre très bien ce qu'on attend d'un projet DevOps : diagnostiquer à partir des logs et corriger proprement via l'automatisation.

5.3 Supervision (infra et applicative)

Le cahier des charges demande de présenter la supervision. Dans notre projet, nous n'avons pas déployé une supervision complète, mais nous avons déjà une base de supervision "technique" par observation et vérification. Sur l'infrastructure, Proxmox donne l'état des VMs (running, io-error, consommation mémoire, stockage). Sur l'applicatif, Nginx permet d'observer les erreurs via `/var/log/nginx/error.log` et de valider la distribution de charge via le contenu de la page d'accueil (hostname). Dans une évolution réaliste, on pourrait ajouter une supervision infrastructure (Prometheus + Node Exporter) et une supervision applicative (Prometheus + Nginx exporter, ou Grafana + dashboards). L'important ici est de montrer qu'on sait où et comment on l'ajouterait, et pourquoi c'est pertinent.

6. Plan de développement et mise en œuvre

La mise en œuvre s'est faite par étapes, car chaque couche dépend de la précédente. Nous avons commencé par stabiliser l'hyperviseur Proxmox et le template Ubuntu, car sans template fonctionnel, Terraform ne peut pas cloner correctement. Ensuite, nous avons écrit la configuration Terraform pour créer les trois VMs, avec cloud-init pour injecter les IP statiques et les clés SSH. Cette phase a permis de valider le provisioning de base : les VMs existent, bootent et sont accessibles.

Une fois les VMs créées, nous avons constaté des problèmes réseau, notamment l'absence d'accès Internet depuis les VMs du réseau interne. Cette étape a été critique car Ansible, pour installer Nginx, dépend du bon fonctionnement d'apt et donc de la connectivité vers les dépôts. Nous avons donc analysé le routage, la passerelle, le NAT, et activé le forwarding IPv4. Après cela, les VMs ont pu joindre Internet (8.8.8.8) et résoudre des noms (deb.debian.org), ce qui a débloqué l'étape Ansible.

En parallèle, nous avons rencontré un incident majeur sur le stockage : le thin pool local-lvm était à 100 %, entraînant des io-error sur des VMs, donc des pertes de connectivité et des comportements incohérents. Nous avons corrigé ce point en augmentant le stockage disponible, puis en étendant le pool LVM thin. Après stabilisation, Terraform et Ansible pouvaient fonctionner correctement.

Enfin, la dernière étape a été la configuration load balancer. Un 502 est apparu, non pas parce que Nginx "ne marchait pas", mais parce que l'upstream pointait vers de mauvaises IP. Le diagnostic s'est fait avec les logs Nginx, puis la correction a été appliquée, ce qui a terminé la chaîne.

Le résultat final est une chaîne complète : Terraform construit, Ansible configure, Nginx sert, et la plateforme est reproductible.

7. Résultats obtenus

À l'issue du projet, nous disposons d'une infrastructure entièrement automatisée permettant de déployer rapidement un environnement applicatif complet. L'ensemble de la solution repose sur les principes de l'Infrastructure as Code et sur l'utilisation d'outils d'automatisation largement utilisés dans les environnements DevOps modernes.

Concrètement, quelques commandes suffisent désormais pour reconstruire l'ensemble de l'environnement. La création des machines virtuelles est assurée automatiquement par Terraform, qui interagit directement avec l'API de Proxmox afin de provisionner les ressources nécessaires. Une fois les machines déployées, Ansible prend le relais pour configurer les serveurs, installer les services et déployer l'application web.

Cette automatisation permet notamment :

- de déployer l'infrastructure complète en quelques minutes
- de configurer automatiquement les serveurs web et le Load Balancer
- de mettre en ligne un service web accessible via une seule adresse IP
- de répartir les requêtes entre plusieurs serveurs grâce au mécanisme de load balancing

Le système mis en place repose sur trois machines virtuelles : un Load Balancer et deux serveurs web. Le Load Balancer, configuré avec Nginx, distribue les requêtes HTTP entre les deux serveurs web, ce qui permet de démontrer concrètement un mécanisme de répartition de charge. Lorsqu'un utilisateur accède à l'adresse du Load Balancer, les requêtes sont redirigées dynamiquement vers l'un des deux serveurs.

Un autre avantage important de cette architecture est sa reproductibilité. L'ensemble de l'infrastructure étant décrit dans des fichiers de configuration, il est possible de recréer l'environnement à l'identique à tout moment. Cette approche permet d'éviter les dérives de configuration entre différents environnements et garantit une cohérence entre les déploiements.

De plus, la configuration Ansible est idempotente, ce qui signifie que relancer le playbook ne provoque pas de modifications inutiles si l'état souhaité est déjà atteint. Cette propriété est particulièrement importante dans les environnements

d'automatisation, car elle permet d'exécuter les scripts plusieurs fois sans risquer de casser la configuration existante.

Ainsi, la solution mise en place permet non seulement de déployer une infrastructure fonctionnelle, mais aussi de démontrer l'intérêt des outils d'automatisation dans un contexte de gestion d'infrastructure moderne.

**MetalOps**
Déploiement continu automatisé (Proxmox + Terraform + Ansible)

Node: vm-web-2

SERVEUR WEB

OK • Nginx

vm-web-2

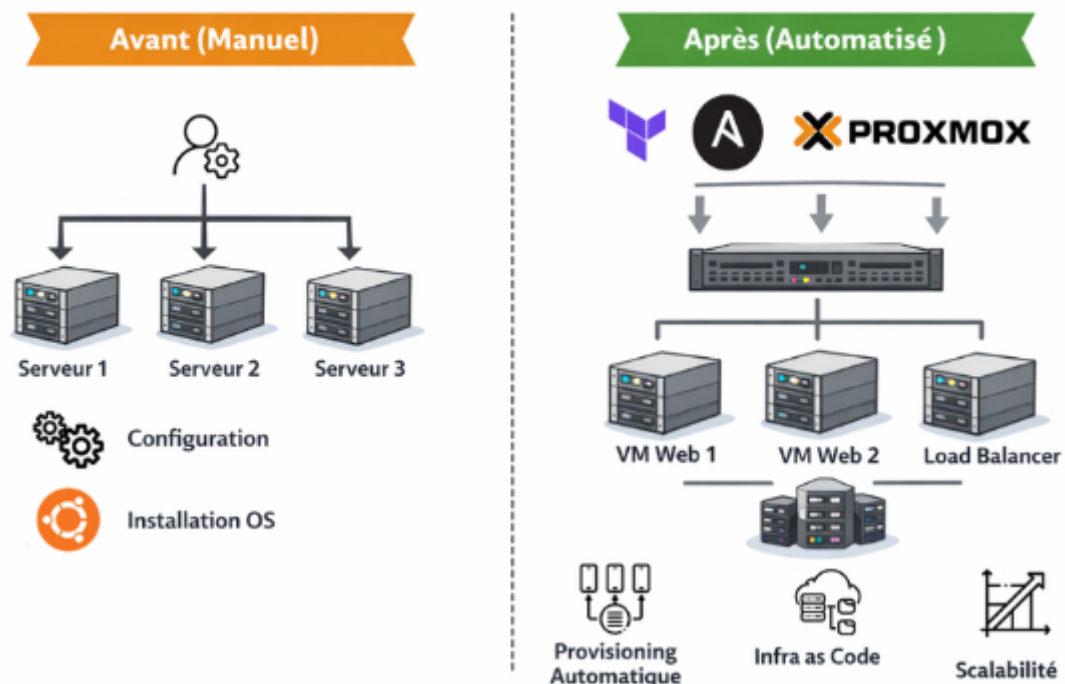
Cette page est générée automatiquement par Ansible. Rafraîchis plusieurs fois via le Load Balancer pour visualiser la répartition.

IP (Ansible)	10.10.10.12
Groupe	web
Date (build)	2026-03-05T18:59:44Z
Accès LB	http://10.10.10.10

MetalOps • Provisioning IaC

Généré par Ansible • 2026-03-05 18:59:44

Comparatif Avant / Après



8. Budget et ressources

Dans le cadre de la SAE, les coûts logiciels sont nuls car tous les outils utilisés sont open source (Proxmox, Terraform, Ansible, Nginx). Le coût principal dans un contexte réel viendrait du matériel (serveurs physiques, stockage, réseau), du temps d'ingénierie pour concevoir et maintenir l'automatisation, et de la supervision/maintenance.

Cependant, l'intérêt économique est justement dans le gain de productivité. Le temps gagné sur chaque déploiement, la réduction des erreurs humaines, et la capacité de reconstruire rapidement un environnement après incident compensent rapidement l'investissement initial. Dans un environnement où les délais sont critiques, c'est souvent ce type de solution qui permet de "reprendre le contrôle" et d'industrialiser les mises en production.

9. Gestion de projet et organisation du travail

Dans le cadre de ce projet, une organisation inspirée des méthodes agiles a été adoptée afin de structurer le développement de la solution. L'objectif était de découper le projet en plusieurs étapes techniques afin de progresser de manière progressive tout en validant régulièrement le fonctionnement de l'infrastructure.

Le travail a été organisé sous forme de backlog de tâches, regroupées par grands objectifs techniques. Chaque objectif correspond à un service technique nécessaire à la mise en place de la solution de déploiement continu.

Le premier objectif consistait à mettre en place l'environnement de virtualisation. Cette étape comprenait l'installation et la configuration de Proxmox, la gestion du stockage et la création d'un template Ubuntu permettant de cloner rapidement de nouvelles machines virtuelles.

La deuxième étape concernait l'automatisation du provisionnement de l'infrastructure à l'aide de Terraform. Les machines virtuelles nécessaires au projet ont été décrites dans des fichiers de configuration afin de permettre leur création automatique via l'API de Proxmox.

Une fois l'infrastructure déployée, la troisième étape a consisté à automatiser la configuration des serveurs à l'aide d'Ansible. Les playbooks ont été développés afin d'installer le serveur web Nginx, déployer une page web et configurer un Load Balancer.

Enfin, une dernière phase de tests a permis de vérifier le bon fonctionnement de l'ensemble de l'architecture. Ces tests ont consisté à valider la connectivité réseau, l'accès Internet des machines virtuelles, ainsi que la répartition des requêtes entre les serveurs web via le Load Balancer.

Cette organisation par étapes a permis de progresser méthodiquement tout en identifiant rapidement les problèmes techniques rencontrés durant le projet.

10. Axes d'amélioration

Bien que la solution mise en place permette de démontrer un déploiement automatisé fonctionnel, plusieurs améliorations pourraient être apportées afin de rendre l'architecture encore plus robuste et plus proche d'un environnement de production réel.

Une première amélioration concernerait la haute disponibilité du Load Balancer. Dans l'architecture actuelle, un seul Load Balancer est utilisé pour distribuer les requêtes vers les serveurs web. Cela signifie que ce composant constitue un point de défaillance unique. Si la machine hébergeant le Load Balancer venait à tomber en panne, l'ensemble du service deviendrait inaccessible.

Une évolution possible consisterait à déployer deux Load Balancers fonctionnant en redondance. Un mécanisme de haute disponibilité, par exemple basé sur Keepalived et une adresse IP virtuelle, permettrait de basculer automatiquement le trafic vers le second Load Balancer en cas de panne du premier. Cette approche permettrait d'améliorer significativement la disponibilité du service.

Une autre amélioration possible serait l'intégration complète de la solution dans un pipeline CI/CD. Actuellement, le déploiement est déclenché manuellement via Terraform et Ansible. Dans un contexte professionnel, ces étapes pourraient être automatisées à l'aide d'un outil d'intégration continue tel que GitLab CI ou Jenkins. Chaque modification de l'infrastructure ou de la configuration pourrait alors déclencher automatiquement un déploiement.

Enfin, il serait également possible d'ajouter des outils de supervision et de monitoring, tels que Prometheus ou Grafana, afin de surveiller l'état des serveurs, l'utilisation des ressources et la disponibilité du service web. Cela permettrait d'anticiper les incidents et d'améliorer la gestion opérationnelle de l'infrastructure.

Ces améliorations permettraient de faire évoluer l'architecture vers une solution encore plus proche des infrastructures utilisées dans les environnements professionnels.

11. Conclusion

Ce projet a permis de mettre en pratique les principes du DevOps et de démontrer l'intérêt de l'automatisation dans le déploiement d'infrastructures informatiques. L'objectif initial était de remplacer des méthodes de déploiement manuelles, longues et sujettes aux erreurs, par une solution automatisée, reproductible et fiable.

Au cours du projet, nous avons conçu et mis en place une architecture reposant sur plusieurs technologies complémentaires. Proxmox a été utilisé comme plateforme de virtualisation afin d'héberger les machines virtuelles nécessaires au projet. Terraform a permis d'automatiser le provisionnement de ces machines, tandis qu'Ansible a été utilisé pour configurer les serveurs et déployer les services applicatifs.

L'un des points les plus intéressants du projet a été la résolution de plusieurs problèmes techniques réels. Nous avons notamment rencontré des difficultés liées au stockage LVM thin, à la configuration du réseau interne, à l'accès Internet depuis les machines virtuelles et à la configuration du Load Balancer. L'analyse et la résolution de ces problèmes nous ont permis de mieux comprendre le fonctionnement interne de nombreux composants système tels que le routage, le NAT, la gestion du stockage ou encore la configuration des services web.

Au-delà de l'aspect technique, ce projet illustre également l'importance des pratiques DevOps dans les environnements professionnels. En automatisant les tâches de déploiement et de configuration, il devient possible de réduire considérablement les délais de mise en production tout en améliorant la fiabilité et la maintenabilité des systèmes.

La solution proposée répond donc pleinement à la problématique initiale de l'entreprise. Elle permet de passer d'une approche artisanale, reposant sur des configurations manuelles, à une approche industrielle basée sur l'automatisation et l'Infrastructure as Code.

Enfin, ce projet démontre que l'automatisation ne constitue pas seulement un gain de confort pour les équipes techniques, mais qu'elle représente aujourd'hui une nécessité pour garantir la rapidité, la stabilité et la reproductibilité des infrastructures modernes.

12. Annexes

Cette annexe présente les commandes utilisées pour reconstruire l'infrastructure et démontrer le fonctionnement de la solution.

L'objectif est de montrer que l'ensemble de l'environnement peut être déployé automatiquement à partir du code.

Avant de commencer la démonstration, il est nécessaire de vérifier que l'hyperviseur Proxmox est démarré et accessible depuis la machine utilisée pour la soutenance.

Une fois connecté à la machine de démonstration, la première étape consiste à se placer dans le dossier contenant la configuration Terraform.

cd SAE-Cloud/terraform

La commande suivante permet de déployer automatiquement les machines virtuelles sur l'hyperviseur Proxmox.

terraform apply

Terraform interroge l'API Proxmox et crée les machines virtuelles définies dans les fichiers de configuration.

Dans notre cas, trois machines sont déployées :

- une machine Load Balancer
- deux machines Web Server

Une fois l'infrastructure créée, la deuxième étape consiste à configurer automatiquement les serveurs à l'aide d'Ansible.

Pour cela, on se place dans le dossier Ansible :

cd ../ansible

Puis on exécute le playbook principal :

ansible-playbook -i inventory.ini sites.yml

Ce playbook réalise automatiquement plusieurs opérations :

- installation du serveur web Nginx
- déploiement de la page web

- configuration du Load Balancer
- démarrage des services

Lorsque le playbook se termine, l'infrastructure est entièrement opérationnelle.

La dernière étape consiste à tester l'accès au service web via le Load Balancer.

curl http://10.10.10.10

Cette commande permet de vérifier que le serveur répond correctement.

Pour démontrer le fonctionnement du Load Balancer, plusieurs requêtes peuvent être envoyées successivement :

for i in {1..6}; do curl http://10.10.10.10; done

Les réponses montrent que les requêtes sont distribuées entre les deux serveurs web.

Cette démonstration permet d'illustrer concrètement le principe de déploiement automatisé mis en place dans le projet.

